Prompt Injection Detection Using Small Language Models

FERNANDO WWAHH

215523H

Bachelor of Science Honours in Artificial Intelligence

Department of Computational Mathematics
Faculty of Information Technology

University of Moratuwa
Sri Lanka

MAY 2026

Prompt Injection Detection Using Small Language Models

FERNANDO WWAHH

215523H

Thesis submitted in partial fulfillment of the requirements for the
Research Project in Artificial Intelligence for the degree
of BSc Hons in Artificial Intelligence

Department of Computational Mathematics
Faculty of Information Technology

University of Moratuwa
Sri Lanka

MAY 2026

DECLARATION

I declare that this is my own work and this thesis/dissertation does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other University or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text. I retain the right to use this content in whole or part in future works (such as articles or books).

Signature:  Date: 05/05/2026

The above candidate has carried out research for the undergraduate thesis/dissertation under my supervision. I confirm that the declaration made above by the student is true and correct.

Name of Supervisor: Prof. A.T.P. Silva

Signature of the Supervisor: Date:

DEDICATION

I dedicate this thesis to my family, who have always stood by me with love and encouragement, and to my supervisor, whose expertise, guidance, and patience made this research possible.

ACKNOWLEDGEMENT

I would like to express my sincere thanks and appreciation to all those who have supported and guided me throughout my academic journey and in completing this research. First and foremost, I extend my heartfelt gratitude to my supervisor, Prof. A.T.P. Silva, for his invaluable guidance, continuous support, encouragement, motivation, and patience throughout my studies at the University of Moratuwa. Her insightful feedback and mentorship have been instrumental in shaping this research.

I would also like to thank all the lecturers and academic staff of the Department of Computational Mathematics, Faculty of Information Technology, University of Moratuwa, for sharing their knowledge and providing a strong academic foundation. My sincere gratitude extends to my friends and colleagues, who supported me throughout this journey with encouragement, collaboration, and meaningful discussions. Finally, I am deeply grateful to my family for their unconditional love, understanding, and continuous support. Their encouragement has been my greatest strength and inspiration throughout this journey.

ABSTRACT

Prompt injection attacks are a growing security issue in Large Language Model (LLM)-integrated applications, where malicious instructions can manipulate a model into ignoring original instructions, leaking hidden prompts, or performing unintended actions. Existing detection methods provide useful protection, but many are either dependent on large models, limited to specific attack types, or costly to deploy in resource-constrained environments. This research proposes a lightweight and modular prompt injection detection approach using a quantized Small Language Model (SLM) as the backbone and multiple category-specific LoRA adapters as specialised detectors.

The proposed system uses Gemma3-1B-it as the baseline SLM, loaded in 4-bit quantized form using QLoRA. Separate LoRA adapters are trained for different prompt injection categories, including role/instruction violation, privilege escalation, and obfuscation/evasion. During runtime, the adapters are swapped sequentially over the shared backbone model, and the prompt is classified as malicious if any adapter detects an injection attempt. This design avoids training multiple full models while supporting modular extension for new attack categories.

Keywords: Small Language Models, Low Rank Adaptation, Quantization, Prompt Injections, Large Language Models, Fine Tuning

TABEL OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Description
LLM	Large Language Model
LoRA	Low-Rank Adaptation
PEFT	Parameter Efficient Fine-Tuning
QLoRA	Quantized Low-Rank Adaptation
SLM	Small Language Model

CHAPTER 1

INTRODUCTION

1.1 Introduction

This chapter introduces the research project and establishes the foundation for the proposed prompt injection detection framework. It first presents the background of Large Language Model (LLM) security and explains why prompt injection has become an important threat in LLM-integrated applications. The chapter then outlines the objectives of the project, defines the problem in brief, and explains the motivation for using Small Language Models (SLMs) as lightweight security classifiers. It also introduces the proposed specialist SLM-based detection approach, summarizes the resource requirements, and presents the structure of the remaining thesis. The main purpose of this chapter is to provide a clear overview of the research context, the practical problem being addressed, and the direction of the proposed solution before the detailed literature review, technology study, approach, design, and implementation chapters are presented.

1.2 Objectives

The primary objectives of this project are:

- To conduct a critical review of existing prompt injection detection techniques, identifying their strengths, limitations, and research gaps.

- To examine Small Language Models and assess their suitability as lightweight security guardrails for LLM-integrated applications

  .

- To design and implement a sequential specialist SLM ensemble for real-time prompt injection detection, using Parameter-Efficient Fine-Tuning (PEFT) techniques.

- To evaluate the proposed system using Precision, Recall, F1-score, and False Positive Rate (FPR), benchmarked against established baseline detection methods.

- To assess the practical deployability of the solution as a lightweight, real-time security guardrail for resource-constrained environments.

1.3 Problem in Brief

Prompt injection attacks are difficult to detect because they appear as natural language rather than executable code. The malicious instruction may be direct, indirect, encoded, role-based, or hidden inside a benign-looking prompt. These attack types have different semantic and structural features, making detection more complex than ordinary text classification.

Many existing approaches treat prompt injection as a single binary classification problem. This limits detection performance because one model must learn a broad decision boundary across many different attack categories. A detector that identifies direct instruction override may fail against encoded payloads, metaphor-based jailbreaks, or fluent adversarial prompts [2], [11]. At the same time, methods that require internal model access or heavy computation are difficult to deploy in real-time systems [1], [3].

Therefore, the main research problem is the lack of a lightweight, black-box-compatible, and category-specialized detection framework that can identify different types of prompt injection attacks while remaining practical for resource-constrained deployment.

1.4 Background and Motivation

The increasing use of LLMs in agentic and retrieval-based applications has created a need for security mechanisms at the input layer. In such systems, the model may process user prompts, retrieved documents, and tool outputs before producing a response or taking an action. If malicious instructions are hidden inside these inputs, the model may follow the attacker's objective instead of the intended task [4].

Existing defenses show that prompt injection cannot be solved by one method alone. Prompt-formatting defenses reduce instruction confusion but depend on application-level control [4], [23]. Attention-based detection provides useful internal signals but is unsuitable for black-box APIs [1]. Trained detectors are more deployable but may be brittle when attacks differ from the training distribution [8], [20]. These limitations motivate a detection approach that is lightweight, specialized, and independent of the protected LLM.

SLMs are suitable for this direction because prompt injection detection is a focused classification task rather than an open-ended generation task. Recent studies show that SLMs can perform well in practical classification and agentic workloads while reducing cost and latency [24], [25], [26], [27]. With LoRA and QLoRA, these models can be fine-tuned efficiently under limited GPU memory [16], [21]. This makes SLMs a practical foundation for building deployable prompt injection guardrails.

1.5 Noval Approach To Prompt Injection Detection

This project proposes a specialist SLM-based framework for prompt injection detection. The main idea is to divide the detection problem into separate attack categories and train dedicated LoRA Adapter for each category and apply it to the backbone SLM. The proposed categories include instruction and role violation, privilege escalation, and obfuscation or evasion.

Each specialist model receives the raw user prompt and produces a binary output. The outputs are combined using a deterministic decision rule. If any specialist identifies an attack, the final decision is INJECTION DETECTED. If all specialists classify the prompt as safe, the final decision is BENIGN.

The proposed framework is designed as a black-box middleware layer. It does not require access to the protected LLM's weights, hidden states, attention maps, logits, or system prompt. This makes it suitable for deployment with both open-source models and proprietary LLM APIs. The novelty of the project lies in combining SLM-based detection, category-wise specialization, and parameter-efficient fine-tuning into a lightweight security guardrail.

1.6 Resource Requirements

All model training and evaluation is conducted on Google Colab Free Tier with an NVIDIA Tesla T4 GPU (~15-16 GB VRAM). Combined with 4-bit NF4 quantisation via BitsAndBytes, this accommodates fine-tuning and inference for models up to 1.5 billion parameters. Local development and data preprocessing are performed on an Intel Core i9 / AMD Ryzen 9 workstation with 16 GB RAM.

The core stack comprises Python 3.12+, HuggingFace Transformers, PEFT, TRL (SFTTrainer), BitsAndBytes, the HuggingFace Datasets library, and Scikit-learn for evaluation metrics. Training data is drawn from publicly available datasets. Model checkpoints are persisted to Google Drive to ensure continuity across Colab sessions.

1.7 Structure of the Thesis

The remainder of this thesis is organised as follows:

- Chapter 2 - Literature Review: Critical analysis of existing prompt injection detection and prevention techniques, culminating in a structured research gap analysis.

- Chapter 3 - Technology: Study of the core technologies underpinning the proposed system, including the Transformer architecture, SLM design principles, and LoRA/QLoRA fine-tuning.

- Chapter 4 - Approach: Conceptual definition of the proposed detection framework, covering hypothesis, inputs, outputs, and the rationale for the specialist ensemble design.

- Chapter 5 - Analysis and Design: System architecture, dataset construction, model selection, and the risk aggregation pipeline design.

- Chapter 6 - Implementation: Realisation of the architecture, detailing training configurations, data pipeline, and Colab session management strategies.

1.8 Summary

This chapter introduced the research area of prompt injection detection and explained the motivation for using a LoRA fine-tuned Small Language Model. The problem was identified as the need for a lightweight and adaptable detector for malicious prompts in LLM-integrated applications. The chapter also stated the research objectives, proposed approach, resource requirements, and structure of the report. The next chapter reviews existing research to identify the technological and research gaps addressed by this project.

CHAPTER 2

PROMPT INJECTION THREATS AND DETECTION APPROACHES IN LARGE LANGUAGE MODEL SYSTEMS

2.1 Introduction

This chapter provides a thematic survey of the foundational research supporting the proposed framework, specifically focusing on the taxonomy and characterization of prompt injection and jailbreak attacks. The literature review evaluates current defensive methodologies, ranging from heuristic and statistical detection approaches to model-based and learned systems, while also exploring the emerging utility of Small Language Models (SLMs) in resource-constrained security contexts and parameter-efficient fine-tuning using LoRA. By synthesizing the relative strengths and limitations of these existing strategies, the discussion culminates in a comparative summary that formally defines the research gap necessitating this work.

2.2 Related Work

Prompt injection attacks occur when malicious instructions are inserted into the input of an LLM-integrated application, causing the model to follow the attacker's objective instead of the intended system or user instruction. Perez and Ribeiro introduced PromptInject and studied attacks such as goal hijacking and prompt leaking, showing that even simple crafted prompts can misalign language model behavior [13]. This work is important because it established prompt injection as a practical security risk in real-world LLM applications. Later studies showed that jailbreak attacks are more diverse and systematic. Liu et al. empirically studied jailbreak prompts against ChatGPT and identified multiple jailbreak patterns and categories, demonstrating that prompt structure strongly affects attack success [15]. Shen et al. analyzed 1,405 jailbreak prompts collected from real-world online communities and showed that jailbreak prompts evolve over time and can remain effective for long periods [9]. These studies show that prompt injection is not a single fixed attack, but a broad and changing class of adversarial behavior. Automated jailbreak generation has also become an important direction. AutoDAN generates stealthy jailbreak prompts using a hierarchical genetic algorithm and shows strong transferability across models while preserving semantic meaningfulness [2]. Yan et al. proposed AVATAR, which uses adversarial metaphors to guide models from benign-looking content toward harmful outputs [11]. These works show that attackers can bypass simple filters by using indirect, optimized, or semantically disguised prompts.

Prompt injection attacks occur when malicious instructions are inserted into the input of an LLM-integrated application, causing the model to follow the attacker's objective instead of the intended system or user instruction. Perez and Ribeiro introduced PromptInject and studied attacks such as goal hijacking and prompt leaking, showing that even simple crafted prompts can misalign language model behavior [13]. This work is important because it established prompt injection as a practical security risk in real-world LLM applications. Later studies showed that jailbreak attacks are more diverse and systematic. Liu et al. empirically studied jailbreak prompts against ChatGPT and identified multiple jailbreak patterns and categories, demonstrating that prompt structure strongly affects attack success [15]. Shen et al. analyzed 1,405 jailbreak prompts collected from real-world online communities and showed that jailbreak prompts evolve over time and can remain effective for long periods [9]. These studies show that prompt injection is not a single fixed attack, but a broad and changing class of adversarial behavior. Automated jailbreak generation has also become an important direction. AutoDAN generates stealthy jailbreak prompts using a hierarchical genetic algorithm and shows strong transferability across models while preserving semantic meaningfulness [2]. Yan et al. proposed AVATAR, which uses adversarial metaphors to guide models from benign-looking content toward harmful outputs [11]. These works show that attackers can bypass simple filters by using indirect, optimized, or semantically disguised prompts.

Small Language Models have gained attention because they can reduce inference cost, latency, and hardware requirements while remaining effective for focused tasks. Schick and Schütze showed that small language models can achieve strong few-shot performance when tasks are reformulated effectively [14]. More recent surveys argue that SLMs in the 1B–8B parameter range can be practical alternatives to large models when performance, efficiency, scalability, and cost are balanced [24], [28], [29]. Several model families demonstrate the practical potential of SLMs. TinyLlama presents a compact 1.1B parameter model trained on a large token corpus and designed using LLaMA-style architecture [30]. MiniCPM introduces 1.2B and 2.4B parameter models that can compete with larger models through scalable training strategies [17]. Gemma 3 provides lightweight open models ranging from 1B to 27B parameters with improved instruction-following and multilingual abilities [12]. These studies show that SLMs are suitable candidates for specialized tasks such as prompt classification and security filtering. Industrial studies also support the use of smaller models for classification. Li et al. evaluated small transformer models in real-world text classification tasks and highlighted their advantages in VRAM usage and local deployment [27]. Sharma and Mehta further argue that SLMs are suitable for agentic systems where structured and task-specific accuracy is more important than open-ended generation [26]. Since prompt injection detection is a focused classification task, SLMs are a suitable technology direction for this research.

Fine-tuning all parameters of a large model is computationally expensive. LoRA addresses this issue by freezing the pretrained model weights and inserting trainable low-rank matrices into transformer layers, significantly reducing the number of trainable parameters [16]. Hu et al. showed that LoRA can reduce trainable parameters by large margins while maintaining comparable model quality and avoiding additional inference latency [16]. This makes LoRA highly relevant for adapting a language model to a specific security task. QLoRA extends this idea by enabling fine-tuning through a frozen 4-bit quantized language model while training LoRA adapters [21]. Dettmers et al. introduced NF4 quantization, double quantization, and paged optimizers to reduce memory usage while preserving fine-tuning performance [21]. S-LoRA focuses on serving many LoRA adapters efficiently by managing adapters and KV cache memory using unified paging [22]. Together, these works show that LoRA-based adaptation is not only useful for training, but also practical for deployment. For this research, LoRA is important because it enables a selected SLM to be adapted for prompt injection detection without the cost of full model fine-tuning. This supports the project goal of building a lightweight and deployable detector.

2.3 Research Gap

The reviewed literature shows that prompt injection attacks are becoming more diverse, automated, and stealthy [2], [9], [11], [15]. Existing defenses provide important solutions, but each has limitations. Prompt-level defenses such as spotlighting and encoding can reduce attack success, but they may not generalize to all application formats [4], [6]. Perplexity-based detection is lightweight, but it is less suitable for natural and semantically meaningful attacks such as AutoDAN or metaphor-based jailbreaks [2], [7], [11]. Larger model-based detectors and adaptive defenses can improve robustness, but they may be costly for practical deployment [3], [20], [31].

At the same time, research on SLMs shows that smaller models can perform well on focused classification tasks while reducing resource requirements [24], [27], [30]. LoRA and QLoRA provide efficient ways to adapt such models without full fine-tuning [16], [21]. However, there is still a research gap in combining these directions into a practical detection approach: a LoRA fine-tuned Small Language Model specifically designed for prompt injection detection with attention to deployability, efficiency, and classification performance.

Therefore, this project addresses the gap by developing a LoRA-adapted Quantized SLM that learns prompt injection patterns from labelled data and evaluates whether this approach can provide an effective and lightweight alternative to existing detection methods.

2.4 Summary

This chapter reviewed existing work on prompt injection attacks, jailbreak methods, detection techniques, Small Language Models, and LoRA-based fine-tuning. The literature shows that prompt injection is a serious and evolving threat to LLM-integrated applications. Existing detection and defense methods include perplexity filtering, prompt transformation, attention tracking, localization, classifier-based detection, and larger model-based defenses. However, these methods can face limitations in generalization, cost, deployment complexity, or adaptability. The review also showed that SLMs and LoRA provide a promising foundation for efficient task-specific model adaptation. The identified research gap motivates the proposed approach of developing a LoRA fine-tuned Small Language Model for prompt injection detection.

CHAPTER 3

THEORETICAL FOUNDATIONS FOR THE EXTENSION

3.1 Introduction

This chapter presents the theoretical foundations that support the proposed research extension. The project focuses on developing a prompt injection detection system using a quantized Small Language Model (SLM) as the backbone and multiple LoRA adapters as specialised detectors for different prompt injection categories. Therefore, this chapter discusses the existing theories behind SLMs, transformer-based language models, parameter-efficient fine-tuning, LoRA, QLoRA, and adapter-based model extension. The purpose of this chapter is not to describe implementation details, but to explain why these technologies are suitable for extending current prompt injection detection methods. The discussion also revisits the research gap identified in the literature review and explains how the proposed extension is theoretically positioned within the AI body of knowledge.

3.2 Overview of Existing Theory

Modern language models are mainly based on the Transformer architecture, which uses attention mechanisms instead of recurrent or convolutional structures. The Transformer introduced self-attention as a method for modelling relationships between tokens in a sequence, allowing language models to process text more effectively and in parallel [17]. This architecture forms the foundation of most current LLMs and SLMs.

Large Language Models have achieved strong performance in natural language tasks, but they are expensive to train, fine-tune, and deploy. This limitation has increased interest in Small Language Models. SLMs aim to provide useful language understanding and task-specific performance with fewer parameters and lower computational requirements. Recent studies show that SLMs can perform effectively in focused tasks such as text classification, structured outputs, and agentic workflows [24], [26], [27], [30].

Prompt injection detection is one such focused task. Unlike open-ended generation, it requires the model to identify whether an input contains malicious instructions. Existing work has treated this problem using different methods, including perplexity-based detection [7], classifier-based detection [8], attention-based tracking [1], prompt transformation methods such as spotlighting [4], and deployable detector benchmarks such as PromptShield [20]. However, many of these methods either focus on specific attack forms or depend on larger models and higher deployment cost.

Parameter-efficient fine-tuning provides another important theoretical foundation. Instead of updating all parameters of a pretrained model, methods such as LoRA update only a small set of additional trainable parameters [16]. QLoRA further reduces memory requirements by loading the frozen base model in 4-bit quantized form while training LoRA adapters [21]. These techniques make it possible to adapt language models for specialised tasks under limited hardware resources.

3.3 Revisiting the Research Gap

The literature review showed that prompt injection attacks are diverse and continuously evolving. Early work identified direct attacks such as goal hijacking and prompt leaking [13]. Later studies showed that jailbreak prompts can be grouped into multiple structural and behavioural categories [15]. Real-world jailbreak analysis further demonstrated that attackers continuously refine prompts and share them through online communities [9]. Automated methods such as AutoDAN and metaphor-based attacks show that adversarial prompts can be stealthy, transferable, and difficult to detect using simple filtering methods [2], [11].

Existing defenses provide useful contributions, but they leave several gaps. Perplexity-based detection is lightweight, but it is less reliable when attacks are semantically meaningful or written in natural language [7]. Prompt engineering defenses such as spotlighting and encoding can reduce some indirect attacks, but they depend on correct formatting and model compliance [4], [6]. Model-based detectors such as PromptShield and UniGuardian improve detection capability, but practical deployment still requires attention to false positives, cost, and adaptability [20], [31].

The key research gap is therefore the lack of a lightweight, modular, and adaptable prompt injection detection approach that can handle multiple attack categories without training or deploying multiple full models. This project addresses that gap by using one quantized SLM backbone with multiple specialised LoRA adapters.

3.4 Rationale for the Extension

The proposed extension is based on the idea that prompt injection detection should be both category-aware and resource-efficient. Prompt injection attacks differ in purpose and structure. For example, instruction override attacks directly ask the model to ignore previous instructions, while prompt leaking attacks attempt to expose hidden system prompts. Privilege escalation attacks try to give the user or model unauthorized authority, while obfuscation attacks hide malicious intent through indirect wording or encoding. A single detector may struggle to learn all these patterns equally well.

A modular LoRA adapter design provides a suitable solution. The quantized SLM backbone supplies general language understanding, while each LoRA adapter

specialises in one attack category. Since the base model remains frozen, new adapters can be added later without retraining the entire model. This follows the LoRA principle of inserting trainable low-rank matrices into transformer layers while keeping the pretrained weights unchanged [16].

The rationale for using QLoRA is memory efficiency. By loading the backbone in 4-bit NF4 quantization, the system reduces the hardware requirement for training and inference while preserving the ability to train LoRA adapters [21].

The rationale for adapter swapping comes from the need to support multiple specialised detectors over one shared model. S-LoRA shows that many LoRA adapters can be served over a shared base model by storing adapters separately and loading the required adapter during inference [22]. This supports the project design where adapters are sequentially swapped during runtime to detect different prompt injection categories.

3.5 Theoretical Foundations

3.5.1 Small Language Models

Small Language Models occupy a specific niche within this landscape: they apply the same architectural principles as frontier LLMs but are designed through careful data curation, efficient training schedules, and architectural optimisation to achieve competitive performance on targeted tasks at a fraction of the computational cost. Zhang et al. [29] identify SLMs as a distinct and increasingly important category within the AI model hierarchy, noting that the performance gap between SLMs and LLMs on domain-specific classification tasks has narrowed substantially as training methodology has matured. Subramanian et al. [24] and Sakib et al. [28] confirm through broad surveys that well-trained SLMs consistently outperform expectations derived from parameter count alone, particularly when fine-tuned on task-specific data. The emergence of compute-optimal training theory which argues that smaller models trained on proportionally more data outperform larger models trained for fewer steps further legitimises the SLM paradigm as a principled engineering choice rather than a resource compromise [17].

Within the AI security domain specifically, SLMs occupy the role of lightweight inference-time guardrails: models that operate on inputs before they reach a primary LLM, performing fast, targeted classification without the latency or infrastructure requirements of the systems they protect. Belcak et al. [25] and Sharma and Mehta [26] position SLMs as the natural architectural choice for agentic security layers where strict latency budgets apply, and Li et al. [27] provide empirical evidence from industrial text classification deployments confirming that SLMs achieve production-grade precision at substantially lower cost than API-based LLMs.

3.5.2 Parameter-Efficient Fine-Tuning: LoRA and QLoRA

Full fine-tuning of a pre-trained language model updates all parameters, requiring GPU memory sufficient to store the model weights, optimiser states, and gradients simultaneously a cost that is prohibitive for models above a few hundred million parameters on consumer hardware. Low-Rank Adaptation (LoRA), proposed by Hu et al. [16], addresses this by decomposing weight updates into low-rank matrices. For a pre-trained weight matrix $W0 \in R^{d \times k}$, LoRA parameterises the update as:

$$W = W0 + \Delta W = W0 + BA$$

where $B \in R^{d \times r}$ and $A \in R^{r \times k}$ with rank $r \ll min(d, k)$. Only A and B are trained; $W0$ remains frozen. This reduces the number of trainable parameters from $d \times k$ to $r(d \times k)$, typically representing less than 1% of total model parameters, while empirically recovering the performance of full fine-tuning on downstream tasks.

QLoRA extends this by loading the frozen base model in 4-bit NF4 (NormalFloat4) quantisation [21], a quantisation data type optimised for the approximately normal weight distributions of pre-trained language models, while computing LoRA adapter updates in 16-bit precision. Double quantisation further reduces the memory footprint by quantising the quantisation constants themselves [21]. In addition to efficient training, LoRA also supports adapter-based deployment, where multiple task-specific LoRA adapters can be trained for the same frozen base model and swapped when required. Instead of storing and loading separate full models for each task, the system only needs to load the shared base model once and dynamically activate the relevant adapter for a given task. S-LoRA demonstrates this idea at serving scale by storing many LoRA adapters in main memory and fetching only the adapters required by current queries into GPU memory, while using unified paging to manage adapter weights and KV cache tensors efficiently [22]. This adapter-swapping capability is important for this project because it allows the same SLM backbone to support different specialised security tasks, such as prompt injection detection, jailbreak detection, or benign prompt classification, without retraining or deploying separate full models for each task.

3.6 Characteristics of the Extension

The proposed extension has several defining characteristics.

The first characteristic is modularity. Each LoRA adapter is responsible for one prompt injection category, such as instruction and role violation, privilege escalation, or obfuscation and evasion. This makes the system easier to extend when new attack categories are identified. Only the LoRA adapter parameters are trained, while the backbone model remains frozen. This reduces training cost compared with full fine-tuning [16].

The second characteristic is memory efficiency. The backbone model is quantized using QLoRA principles, reducing the memory required to load and adapt the model [21].

The third characteristic is sequential category detection. During runtime, the system swaps adapters sequentially and checks the input prompt against each specialised detector. If one adapter detects malicious intent, the system can return an injection detection result. If all adapters classify the input as benign, the prompt is treated as benign.

The forth characteristic is scalability. Since adapters are smaller than full model copies, additional attack categories can be supported by training and storing additional adapters. This is consistent with scalable LoRA-serving ideas such as S-LoRA [22].

The fifth characteristic is interpretability at category level. Since each adapter corresponds to a known threat category, the system can indicate which type of prompt injection was detected, rather than only producing a general malicious label.

3.7 Bridging the Research Gap with Extension

The proposed extension bridges the research gap by combining three previously separate directions: prompt injection detection, SLM-based efficient deployment, and LoRA-based modular adaptation.

The gap in existing prompt injection detection is that many methods are either too general, too expensive, or too dependent on fixed prompt formats. The proposed system addresses this by using specialised adapters for different attack categories. This makes the detector more aligned with the threat taxonomy identified in the literature [9], [13], [15].

The gap in deployment efficiency is addressed by using a quantized SLM instead of a large model. SLM research supports the use of smaller models for focused classification and deployment-sensitive tasks [24], [27], [30]. QLoRA further reduces the memory requirement by allowing the frozen model to be loaded in 4-bit precision [21].

The gap in adaptability is addressed through LoRA adapters. If a new attack type appears, the system can train a new adapter and add it to the detection sequence without modifying the full backbone model. This makes the proposed extension more maintainable than a single static detector.

Therefore, the extension directly supports the research objective: to create a lightweight and adaptable prompt injection detection system using a quantized SLM backbone and category-specific LoRA adapters.

3.8 Summary

This chapter presented the theoretical foundations for the proposed extension. It reviewed the existing theory behind transformer models, SLMs, prompt injection detection, LoRA, QLoRA, and multiple adapter serving. The research gap was revisited as the need for a lightweight, modular, and adaptable prompt injection detector. The chapter then justified the proposed extension and explained how category-specific LoRA adapters over a quantized SLM backbone can bridge the gap. The next chapter presents the proposed approach based on these theoretical foundations.

CHAPTER 4

APPROACH

4.1 Introduction

This chapter presents the proposed approach for detecting prompt injection attacks using a quantized Small Language Model (SLM) with multiple LoRA adapters. The approach is based on the idea that prompt injection attacks are not a single uniform threat, but a set of different attack categories with different linguistic patterns, intentions, and execution strategies. Instead of training one general detector for all attack types, this research proposes a modular adapter-based detection architecture. A quantized SLM is used as the frozen backbone model, and separate LoRA adapters are trained for different prompt injection categories.

4.2 Hypothesis and its inspiration

The central hypothesis of this work is that decomposing prompt injection detection into three specialised binary classification tasks, each targeting a structurally and semantically distinct attack category, and combining their outputs through a sequential OR-gate decision logic will yield higher aggregate detection performance than a single generalised classifier trained across all attack types simultaneously. This hypothesis rests on two supporting propositions. First, the internal representational demands of detecting role-override attacks, privilege escalation attempts, and encoding-based obfuscation are sufficiently different that a single model must compromise across all three to minimise a shared loss function, whereas a specialist can optimise exclusively for its assigned threat surface. Second, the OR-gate logic reflects the asymmetric cost structure of security classification where a missed attack is substantially more harmful than a blocked benign prompt and operationalises this asymmetry by treating any single specialist's positive detection as sufficient grounds for blocking, without requiring corroboration from the remaining models and without introducing any additional trainable component, provided that each specialist's detection signal is reliable within its own domain an assumption that holds when specialists are trained on category-disjoint label distributions derived from the same raw corpus.

4.3 Inputs and Outputs of Extension

The main input to the extension is a text prompt. This prompt may be a direct user instruction, a conversation message, or a text segment retrieved from an external source. In LLM-integrated systems, such inputs may contain malicious instructions that attempt to override the original system goal, reveal hidden prompts, escalate privileges, or hide harmful intent through obfuscation.

The system produces two main outputs. The first output is a binary decision: Injection Detected or Benign. The second output is the detected injection category, when applicable. For example, if the instruction and role violation adapter detects the attack, the system can identify the prompt as belonging to that category. This category-level output improves interpretability because the system does not only report that a prompt is malicious, but also indicates the likely nature of the attack.

4.4 Process Workflow for Extension

The proposed workflow begins by loading the quantized SLM backbone once. The backbone model is kept frozen and loaded using low-bit quantization to reduce memory usage. Gemma 3 is considered suitable for this role because it provides lightweight open models with strong instruction-following capability [12].

After the backbone is loaded, the fine-tuned LoRA adapters are prepared. Each adapter corresponds to one prompt injection category. Based on the current design, the main adapter categories are:

Table 4.1: LoRA Adapters and their purposes

Adapter	Purpose
Instruction and Role Violation Adapter	Detects prompts that attempt to override instructions or force the model into an unauthorized role
Privilege Escalation Adapter	Detects prompts that attempt to gain higher authority or bypass permission boundaries
Obfuscation and Evasion Adapter	Detects prompts that hide malicious intent through indirect wording, encoding, or disguised phrasing

During runtime, the user prompt is first evaluated using the Instruction and Role Violation adapter. If the adapter classifies the prompt as malicious, the system immediately returns Injection Detected. If the prompt is classified as benign, the system swaps to the next adapter and continues the detection process. The same flow is repeated with the Privilege Escalation adapter and then the Obfuscation and Evasion adapter.

This creates a sequential detection pipeline. The prompt is only classified as benign if all adapters return benign results. This workflow is consistent with the adapter-swapping concept, where multiple LoRA adapters can be served over one base model instead of deploying separate full models for each task [22].

The workflow can be summarized as:

1. Load quantized SLM backbone.
2. Load or prepare category-specific LoRA adapters.
3. Receive user prompt.
4. Activate first adapter and perform detection.
5. If malicious, return injection detected.
6. If benign, swap to the next adapter.
7. Repeat until all adapters are checked.
8. Return benign only if no adapter detects an injection.

4.5 Technologies Identified

The main technologies identified for this research are SLMs, Gemma 3, LoRA, QLoRA, and adapter-based serving.

The selected backbone technology is a Small Language Model. SLMs are suitable because the task is a focused classification problem rather than open-ended generation. Prior studies show that SLMs can be effective in practical text classification and deployment-sensitive environments [24], [27], [30]. The selected model family is Gemma 3. Gemma 3 provides lightweight open models ranging from 1B to 27B parameters, with instruction-following, multilingual, and long-context capabilities [12]. The 1B instruction-tuned variant is suitable for a resource-constrained research setting.

The selected fine-tuning method is LoRA. LoRA freezes the pretrained model and trains only low-rank adapter matrices, reducing trainable parameters and memory cost [16]. This is suitable for training separate adapters for different prompt injection categories. The selected memory-efficiency method is QLoRA. QLoRA loads the frozen base model in 4-bit quantized form and trains LoRA adapters on top of it, reducing GPU memory requirements [21].

The selected deployment concept is multiple LoRA adapter swapping. S-LoRA demonstrates that many LoRA adapters can be stored and served over a shared backbone model efficiently [22]. This supports the proposed runtime design where adapters are swapped sequentially for category-wise detection.

4.6 Features of Technological Extension

The proposed technological extension has several main features. First, it is modular. Each LoRA adapter is responsible for one injection category. This means the system can be extended by adding new adapters when new attack types are identified.

Second, it is resource-efficient. The use of a quantized SLM backbone reduces memory usage, while LoRA reduces the number of trainable parameters [16], [21].

Third, it is category-aware. Instead of only producing a general malicious label, the system can identify the adapter category that triggered the detection. Fourth, it supports adapter swapping. The same backbone model can be reused with different adapters at runtime. This avoids storing and loading multiple full models [22]. Fifth, it is adaptable. If one attack category changes over time, only the relevant adapter needs to be retrained or updated. The frozen backbone and other adapters can remain unchanged. Sixth, it is deployment-oriented. The approach is designed with practical resource limits in mind, making it more suitable for local or lightweight security filtering than large-model-based detection systems.

4.7 Target Users / Use Scenarios

The main target users of this extension are developers and researchers building LLM-integrated applications. These include chatbot developers, AI application builders, security researchers, and organizations that use LLMs with external data sources. A key use scenario is pre-processing user prompts before they are sent to an LLM. In this scenario, the proposed detector acts as a guard layer that checks whether a prompt contains malicious instructions. Another use scenario is retrieval-augmented generation systems. In RAG applications, retrieved documents may contain indirect prompt injection instructions. The detector can be used to screen retrieved text before it is passed into the final LLM prompt [4]. A third use scenario is AI security testing. Researchers can use the system to test different prompt injection categories and evaluate how well category-specific adapters detect them. A fourth use scenario is modular enterprise deployment. An organization may train separate adapters for its own security policies and swap them depending on the application context.

4.8 Positioning the Extension within the AI Body of Knowledge

This research is positioned at the intersection of AI security, Small Language Models, and parameter-efficient fine-tuning. Within AI security, the research contributes to prompt injection detection, which is a major problem in LLM-integrated applications. It builds on prior work on prompt injection attacks, jailbreak attacks, and detection mechanisms. Within SLM research, the project supports the view that smaller models can be effective for specialised and deployment-sensitive tasks. This aligns with recent studies arguing that SLMs are suitable for task-specific classification, agentic systems, and efficient deployment. Within parameter-efficient fine-tuning, the research applies LoRA and QLoRA to a security detection task. LoRA provides the mechanism for lightweight task adaptation [16], while QLoRA enables adaptation under limited hardware resources [21]. The use of multiple adapter swapping extends this further by making the system

modular and scalable [22]. Therefore, the proposed extension contributes a practical design pattern: a quantized SLM backbone with sequential category-specific LoRA adapters for prompt injection detection.

4.9 Summary

This chapter presented the proposed approach for the research. The hypothesis states that a quantized SLM with category-specific LoRA adapters can provide a modular and resource-efficient method for prompt injection detection. The chapter defined the inputs and outputs of the extension, described the sequential adapter-swapping workflow, identified the required technologies, and explained the main features of the technological extension. It also discussed target users and positioned the research within AI security, SLMs, and parameter-efficient fine-tuning. The next chapter converts this approach into the analysis and design of the proposed system.

CHAPTER 5

ANALYSIS AND DESIGN

5.1 Introduction

This chapter presents the analysis and design of the proposed prompt injection detection system. The conceptual approach described in the previous chapter is converted into a top-level system architecture without discussing implementation-level algorithms. The design follows the project direction of using a quantized Small Language Model (SLM) as a shared backbone and multiple category-specific LoRA adapters for detecting different types of prompt injection attacks. The proposed architecture is designed around three main ideas. First, a lightweight SLM provides the general language understanding needed for prompt classification. Second, QLoRA-based quantization reduces memory requirements by loading the frozen backbone model in low-bit precision [21]. Third, separate LoRA adapters are used for different attack categories, allowing the system to specialize without training or deploying multiple full models [16], [22].

5.2 Rationale for the design of Extension

The proposed architecture is consistent with the research problem because prompt injection attacks are category-dependent rather than uniform. Existing research shows that attacks can differ significantly in structure and intent. Perez and Ribeiro identified goal hijacking and prompt leaking as early forms of prompt injection [13]. Liu et al. showed that jailbreak prompts can be grouped into different patterns and categories [15]. AutoDAN further demonstrates that attacks can be automatically generated in stealthy and transferable forms [2]. Therefore, using a single general detector may not capture all attack-specific patterns equally well.

The adapter-based architecture addresses this issue by decomposing detection into multiple specialized stages. Each LoRA adapter focuses on one category of malicious behavior. This improves architectural consistency because the system design directly follows the threat taxonomy defined in the approach chapter. Instruction and role violation detection targets prompts that attempt to override system instructions or force a new role. Privilege escalation detection targets prompts that attempt to gain unauthorized authority over the system. Obfuscation and evasion detection targets prompts that hide malicious intent through encoding, indirect wording, or disguised phrasing.

The architecture is also consistent with the resource constraints of the project. Full fine-tuning of a model for each attack type would require separate copies of model weights, which is inefficient. LoRA avoids this by freezing the base model and

training only small adapter parameters [16]. QLoRA further reduces memory usage through 4-bit quantization of the frozen backbone [21]. S-LoRA shows that many LoRA adapters can be served over a shared base model by loading and managing adapters separately from the backbone [22]. Therefore, the proposed architecture follows a scalable design where the base model is loaded once and adapters are swapped sequentially during detection.

The design is also suitable for future extension. If new prompt injection categories are identified, the system does not need to retrain the full backbone. A new LoRA adapter can be trained for the new category and added to the detection pipeline. This makes the architecture flexible and maintainable as attack strategies evolve. It also supports comparative evaluation because each adapter can be tested independently and as part of the full sequential pipeline. Finally, the architecture maintains a clear separation between model backbone, adapter modules, detection flow, and output decision. This separation improves system clarity and supports the next implementation stage, where each component can be developed and tested independently.

5.3 Top-Level Architecture

Figure 5.1 shows the top-level architecture of the proposed system. The architecture is divided into three major stages: initial loading, detection process, and output generation.

The first stage is the initial loading stage. In this stage, the SLM backbone is loaded once into memory. The selected backbone is a quantized instruction-tuned SLM, such as Gemma3-1B-it, using QLoRA 4-bit NF4 quantization. Gemma 3 is suitable for this role because it provides lightweight open models with improved instruction-following capabilities [12]. Quantization reduces memory consumption while keeping the base model usable for downstream adaptation [21].

Alongside the backbone model, the fine-tuned LoRA adapters are also prepared. In the current design, the architecture contains three category-specific adapters:

1. **Instruction and Role Violation Detection Adapter**
2. **Privilege Escalation Detection Adapter**
3. **Obfuscation and Evasion Detection Adapter**

Each adapter is trained for one prompt injection category. This design follows the LoRA principle where the pretrained model remains frozen and only small low-rank matrices are trained for downstream adaptation [16]. Since each adapter is lightweight, multiple adapters can be stored and swapped without duplicating the full model.

Figure 5.1: Top-level architecture

The second stage is the runtime detection process. When a user prompt is received, the system attaches the first LoRA adapter to the SLM backbone and performs category-specific detection. If the adapter detects the prompt as malicious, the system immediately returns an injection detection result. If the prompt is classified as benign by the first adapter, the system swaps to the next adapter and repeats the detection process. This sequential flow continues through the privilege escalation adapter and then the obfuscation and evasion adapter. If any adapter identifies the input as malicious, the final result is Injection Detected. If all adapters classify the prompt as benign, the final result is BENIGN.

The third stage is the output stage. The output is a binary detection result: either malicious or benign. In the extended design, the system can also return the adapter category that triggered the detection, which supports interpretability by indicating the most likely injection type.

This architecture supports modular detection because each adapter represents a separate security capability. New categories can be added by training additional

LoRA adapters and inserting them into the runtime detection sequence. This aligns with recent work showing that prompt injection attacks are diverse and include instruction override, prompt leaking, indirect injection, privilege manipulation, and stealthy jailbreak patterns [2], [9], [13], [15].

5.4 Data Flow and Interaction Design

The data flow begins with the user prompt. The prompt is passed into the detection system before it reaches the downstream LLM application. This makes the proposed extension function as a protective guard layer for LLM-integrated systems.

The first interaction occurs between the user prompt and the SLM backbone with the Instruction and Role Violation adapter. This adapter checks whether the prompt attempts to ignore previous instructions, redefine the model's role, or bypass instruction boundaries. If malicious behaviour is detected, the interaction ends and the system returns Injection Detected.

If the first adapter returns benign, the same prompt flows to the SLM backbone with the Privilege Escalation adapter. This checks whether the prompt attempts to gain higher authority, request unauthorized control, or manipulate permission levels. Again, a malicious result stops the process and produces the final injection warning.

If the second adapter returns benign, the prompt flows to the SLM backbone with the Obfuscation and Evasion adapter. This checks whether the prompt hides malicious intent through indirect wording, encoded text, misleading formatting, or evasion strategies. If this adapter also returns benign, the final output is BENIGN.

The interaction design therefore follows a sequential decision flow:

Table 5.1: Interactions and decisions in the detection flow

Step	Interaction	Decision
1	User prompt enters the detector	Prompt is prepared for category checking
2	Adapter 1 evaluates instruction and role violation	Malicious → Injection Detected; Benign → next adapter
3	Adapter 2 evaluates privilege escalation	Malicious → Injection Detected; Benign → next adapter

4	Adapter 3 evaluates obfuscation and evasion	Malicious → Injection Detected; Benign → final benign output
5	Final output is generated	Injection Detected or BENIGN

This flow is consistent with the threat taxonomy from the approach chapter. It also supports efficient resource use because the same quantized backbone is reused throughout the process, and only the active LoRA adapter changes.

5.5 Extension Integration into AI Framework

The proposed extension is intended to be integrated into an LLM-based AI application as a pre-inference security layer. Before a prompt is passed to the main application model, it is first processed by the quantized SLM detector. If the detector classifies the prompt as benign, the prompt can continue to the downstream application. If the detector identifies an injection attempt, the application can block the request, log the event, or request user clarification.

Within an AI framework, the extension can be positioned between the input interface and the main LLM or agent. In a chatbot, it can screen direct user messages. In a retrieval-augmented generation system, it can screen retrieved document chunks before they are inserted into the final prompt. This is important because indirect prompt injection attacks exploit external content that is concatenated with user instructions [4].

The integration design is modular. The backbone model remains unchanged, while the LoRA adapters represent the extendable security components. If a new attack type appears, a new adapter can be trained and added to the sequence without replacing the existing backbone or retraining all adapters. This makes the extension suitable for continuous improvement as prompt injection techniques evolve.

The design also aligns with parameter-efficient model adaptation. LoRA allows task-specific extensions to be added to a pretrained model without full fine-tuning [16], QLoRA supports memory-efficient use of the backbone [21], and multi-adapter serving concepts support scalable adapter management [22]. Therefore, the extension can be integrated into AI systems as a lightweight, adaptable, and category-aware detection component.

5.4 Summary

his chapter presented the analysis and design of the proposed prompt injection detection system. The top-level architecture uses a quantized Gemma 3 SLM backbone with multiple category-specific LoRA adapters. The detection process is

performed by sequentially swapping adapters and evaluating the user prompt against each attack category. The rationale for the design was explained in relation to prompt injection diversity, resource efficiency, modularity, and scalability. The next chapter will describe how this architecture is implemented using selected software tools, hardware resources, and fine-tuning procedures.

CHAPTER 6

IMPLEMENTATION

6.1 Introduction

This chapter describes the current implementation progress of the proposed prompt injection detection system. The implementation follows the design presented in Chapter 5, where a quantized Small Language Model (SLM) is used as a shared backbone and multiple LoRA adapters are trained for separate injection categories. At the current stage, three datasets have been created for the three detection tasks by combining and preprocessing multiple Hugging Face datasets. Two LoRA adapters have been fine-tuned using Gemma3-1B-it as the baseline SLM: one for role/instruction violation detection and one for privilege escalation detection. Runtime LoRA adapter swapping has also been tested using example adapters from Hugging Face. The remaining work is to fine-tune the third adapter and integrate all adapters into the complete sequential detection pipeline.

6.2 Overall Development Environment

The development environment is based on Python notebook-based experimentation. The implementation uses the Hugging Face ecosystem for dataset management, tokenizer loading, model loading, training, evaluation, and adapter publishing. The training setup uses Unsloth with QLoRA to load Gemma3-1B-it in 4-bit format and fine-tune LoRA adapters efficiently. This follows the theoretical foundation of LoRA, where only low-rank adapter parameters are trained while the base model remains frozen , and QLoRA, where the base model is quantized to reduce memory requirements. The development environment is a Python-based machine learning environment with GPU acceleration.The project uses Python 3.10 or above as the main programming language. PyTorch is used as the main deep learning framework for model training and inference. Hugging Face Transformers is used for loading the Gemma3-1B-it model

6.3 Hardware and Software Platforms

The implementation is designed for GPU-based development environments such as Kaggle or Google Colab.These platforms are suitable because QLoRA reduces memory requirements and allows fine-tuning on commonly available GPUs such as NVIDIA T4 or P100.The development environment uses Python as the main programming language.The baseline Small Language Model used in the implementation is Gemma3-1B-it.The quantized model checkpoint used for training is unsloth/gemma-3-1b-it-unsloth-bnb-4bit.QLoRA is used as the fine-tuning method

to support low-memory training.LoRA is used as the adapter method for training category-specific detector modules.

The main libraries used are Hugging Face Datasets, Transformers, PEFT, TRL, Accelerate, BitsAndBytes, and Unsloth.Hugging Face Datasets is used for loading, combining, processing, and splitting the datasets.Hugging Face Transformers is used for model loading, tokenizer handling, and inference.PEFT is used to create and manage LoRA adapters.TRL and Accelerate are used to support supervised fine-tuning and efficient GPU-based training.BitsAndBytes is used to enable 4-bit quantized model loading.Unsloth is used to optimize QLoRA fine-tuning and reduce training memory usage.Hugging Face Hub is used for hosting the prepared datasets.

Hugging Face Hub is also used for storing and loading the fine-tuned LoRA adapters.Scikit-learn metrics are used for evaluation, including accuracy, precision, recall, F1-score, and confusion matrix.Gemma 3 is used because it provides lightweight open models suitable for instruction-following and efficient adaptation. LoRA and QLoRA are used because they support efficient fine-tuning and deployment under limited hardware resources.

6.4 Module-wise Implementation

6.4.1 Dataset Preparation Module

The dataset preparation module creates three datasets corresponding to the three detection tasks. Multiple Hugging Face datasets are loaded and filtered based on injection categories. Since the source datasets use different column names and label formats, a standardisation function is used to normalize all dataset pieces into two main columns: text and label.

After standardisation, injection samples are grouped into three task-specific datasets. Benign samples are then collected from multiple sources and added to each dataset. The dataset preparation process balances each dataset using a 3:1 benign-to-injection ratio. This ratio was selected to expose the model to more normal prompts while still preserving enough malicious examples for learning.

The notebook also performs exact-text deduplication to reduce repeated samples. After merging and balancing, each dataset is converted into a Gemma-style chat format. Prompt-template variation is used to reduce template overfitting. Finally, each dataset is split into train, validation, and test sets using an 80/10/10 stratified split and pushed to the Hugging Face Hub.

6.4.2 Fine-Tuning Module

The fine-tuning module trains one specialist LoRA adapter at a time. The notebook uses a configuration variable, SELECTED_SLM, to choose which dataset and

adapter are used for a run. For example, selecting "A" trains the role/instruction violation detector, while selecting "B" trains the privilege escalation detector. The model is loaded using the Unsloth 4-bit Gemma 3 checkpoint: unsloth/gemma-3-1b-it-unsloth-bnb-4bit

The backbone model is loaded in 4-bit mode, and LoRA adapters are attached to selected transformer modules such as attention projection layers and feed-forward projection layers. The target modules include q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, and down_proj.

This design follows LoRA's principle of adapting the model through small trainable matrices rather than updating the full model. It also follows QLoRA's low-memory training approach by using a quantized frozen backbone.

6.4.3 Adapter Swapping Test Module

Adapter swapping is a key part of the final system design. At the current stage, runtime adapter swapping has been tested using example adapters from Hugging Face. This test confirmed that different LoRA adapters can be loaded and activated on the same base model during runtime.

This module is important because the final system depends on sequentially activating the role/instruction violation adapter, privilege escalation adapter, and obfuscation/evasion adapter. The ability to swap adapters supports the multi-adapter design described in S-LoRA, where many adapters can be served over a shared base model rather than deploying multiple full models.

6.4.4 Final Detection Pipeline Module

The final detection pipeline is not fully integrated yet. The planned pipeline will load the quantized Gemma3-1B-it backbone once, load all trained LoRA adapters, and then evaluate each user prompt sequentially through the adapters.

The planned detection order is:

1. Role / instruction violation detection adapter
2. Privilege escalation detection adapter
3. Obfuscation and evasion detection adapter

If any adapter detects malicious intent, the system will return Injection Detected. If all adapters return benign, the system will return BENIGN.

6.5 Workflow Diagrams

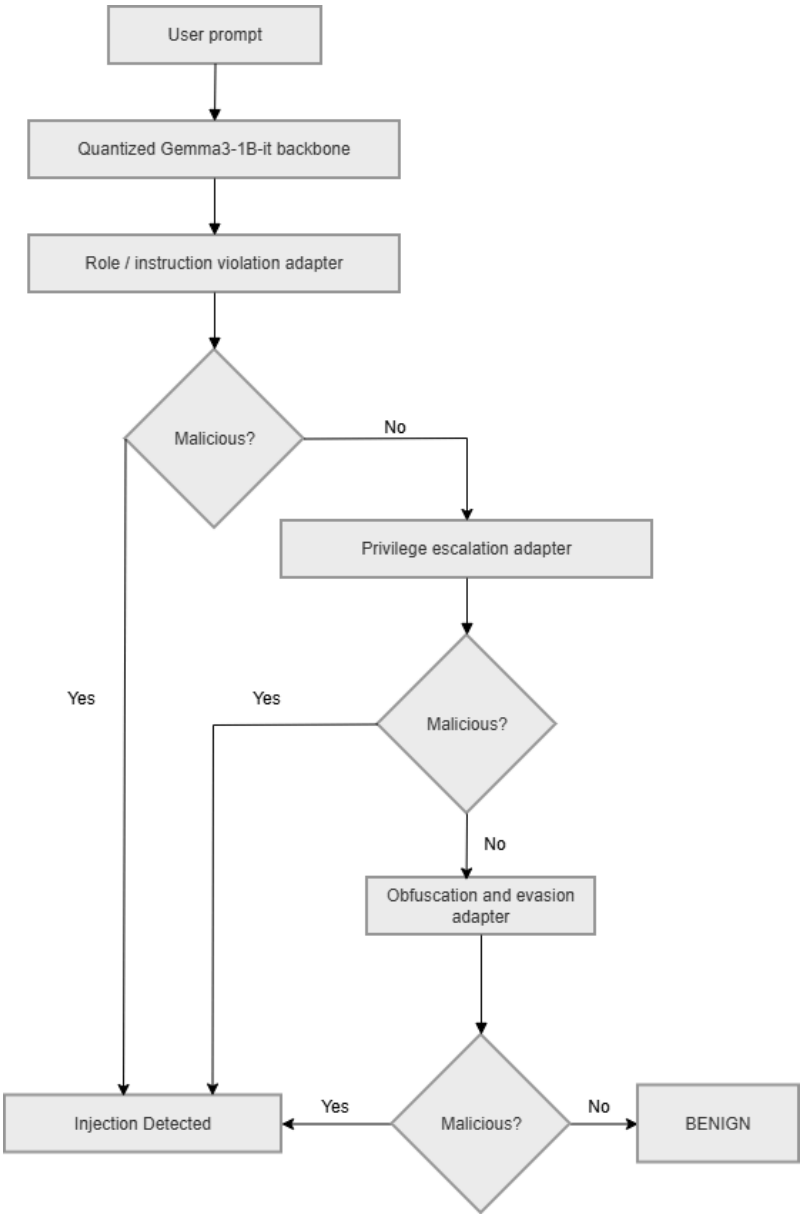

Figure 6.1:

6.6 Integration of Components

The integration stage combines the dataset preparation, fine-tuned adapters, quantized backbone model, and runtime detection pipeline. At present, the dataset preparation module and two fine-tuned adapters have been completed. Adapter swapping has also been tested separately. The remaining work is to fine-tune the third adapter and integrate all trained adapters into one sequential detector.

The final integrated system will load the Gemma3-1B-it quantized backbone once. Then it will load the trained LoRA adapters from storage or from the Hugging Face Hub. During inference, the system will activate each adapter one by one and run the input prompt through the active adapter. This avoids loading multiple full models and supports the modular architecture proposed in Chapter 5.

6.7 Testing During Development

Testing has been performed at different stages of development. During dataset preparation, the datasets were checked for column consistency, label consistency, duplicate prompts, class balance, and train-validation-test split correctness. This ensured that all three datasets were suitable for supervised fine-tuning.

During fine-tuning, the training notebook included evaluation metrics such as accuracy, precision, recall, F1-score, confusion matrix, and classification report. These metrics are used to assess whether each specialist adapter can correctly distinguish injection prompts from benign prompts. The use of precision and recall is important because false positives can block benign prompts, while false negatives can allow malicious prompts to reach the downstream LLM.

Adapter swapping was also tested during development using example Hugging Face adapters. This test was used to confirm the feasibility of changing active LoRA adapters at runtime over a shared base model. The successful test supports the design assumption that the final system can sequentially activate multiple trained adapters during detection.Further testing is still required after the third adapter is fine-tuned and the full pipeline is integrated. The final tests should evaluate the complete system using all three adapters together and compare the sequential detector against individual adapter performance.

6.8 Summary

This chapter described the current implementation progress of the proposed system. Three task-specific datasets have been created by combining and preprocessing Hugging Face datasets. Two LoRA adapters have been fine-tuned using Gemma3-1B-it as the baseline SLM, and runtime adapter swapping has been tested using example adapters. The implementation uses Python notebooks, Hugging Face tools, Unsloth, QLoRA, and LoRA. The remaining work is to fine-tune the obfuscation and evasion adapter and integrate all adapters into the complete sequential prompt injection detection pipeline.

References

[1] K.-H. Hung, C.-Y. Ko, A. Rawat, I.-H. Chung, W. H. Hsu, and P.-Y. Chen, "Attention Tracker: Detecting Prompt Injection Attacks in LLMs," Apr. 23, 2025, arXiv: arXiv:2411.00348. doi: 10.48550/arXiv.2411.00348.

[2] X. Liu, N. Xu, M. Chen, and C. Xiao, "AutoDAN: Generating Stealthy Jailbreak Prompts on Aligned Large Language Models," Mar. 20, 2024, arXiv: arXiv:2310.04451. doi: 10.48550/arXiv.2310.04451.

[3] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong, "DataSentinel: A Game-Theoretic Detection of Prompt Injection Attacks," Nov. 12, 2025, arXiv: arXiv:2504.11358. doi: 10.48550/arXiv.2504.11358.

[4] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending Against Indirect Prompt Injection Attacks With Spotlighting," Mar. 20, 2024, arXiv: arXiv:2403.14720. doi: 10.48550/arXiv.2403.14720.

[5] Y. Chen, H. Li, Z. Zheng, Y. Song, D. Wu, and B. Hooi, "Defense Against Prompt Injection Attack by Leveraging Attack Techniques," Aug. 02, 2025, arXiv: arXiv:2411.00459. doi: 10.48550/arXiv.2411.00459.

[6] R. Zhang, D. Sullivan, K. Jackson, P. Xie, and M. Chen, "Defense against Prompt Injection Attacks via Mixture of Encodings," Apr. 10, 2025, arXiv: arXiv:2504.07467. doi: 10.48550/arXiv.2504.07467.

[7] G. Alon and M. Kamfonas, "Detecting Language Model Attacks with Perplexity," Nov. 07, 2023, arXiv: arXiv:2308.14132. doi: 10.48550/arXiv.2308.14132.

[8] S. Shaheer, G. M. R. Islam, M. R. Hamid, M. A. F. Khan, M. O. Faruk, and Y. Nur, "Detecting Prompt Injection Attacks Against Application Using Classifiers," Dec. 14, 2025, arXiv: arXiv:2512.12583. doi: 10.48550/arXiv.2512.12583.

[9] X. Shen, Z. Chen, M. Backes, Y. Shen, and Y. Zhang, "'Do Anything Now': Characterizing and Evaluating In-The-Wild Jailbreak Prompts on Large Language Models," May 15, 2024, arXiv: arXiv:2308.03825. doi: 10.48550/arXiv.2308.03825.

[10] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and Benchmarking Prompt Injection Attacks and Defenses," Nov. 12, 2025, arXiv: arXiv:2310.12815. doi: 10.48550/arXiv.2310.12815.

[11] Y. Yan et al., “from Benign import Toxic: Jailbreaking the Language Model via Adversarial Metaphors,” in Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2025, pp. 4785–4817. doi: 10.18653/v1/2025.acl-long.238.

[12] G. Team et al., “Gemma 3 Technical Report,” Mar. 25, 2025, arXiv: arXiv:2503.19786. doi: 10.48550/arXiv.2503.19786.

[13] F. Perez and I. Ribeiro, “Ignore Previous Prompt: Attack Techniques For Language Models,” Nov. 17, 2022, arXiv: arXiv:2211.09527. doi: 10.48550/arXiv.2211.09527.

[14] T. Schick and H. Schütze, “It’s Not Just Size That Matters: Small Language Models Are Also Few-Shot Learners,” Apr. 12, 2021, arXiv: arXiv:2009.07118. doi: 10.48550/arXiv.2009.07118.

[15] Y. Liu et al., “Jailbreaking ChatGPT via Prompt Engineering: An Empirical Study,” Mar. 10, 2024, arXiv: arXiv:2305.13860. doi: 10.48550/arXiv.2305.13860.

[16] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” Oct. 16, 2021, arXiv: arXiv:2106.09685. doi: 10.48550/arXiv.2106.09685.

[17] S. Hu et al., “MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies,” Jun. 03, 2024, arXiv: arXiv:2404.06395. doi: 10.48550/arXiv.2404.06395.

[18] J. Pan, S. L. Wong, Y. Yuan, and X. W. Chia, “Prompt Inject Detection with Generative Explanation as an Investigative Tool,” Feb. 16, 2025, arXiv: arXiv:2502.11006. doi: 10.48550/arXiv.2502.11006.

[19] Y. Jia, Y. Liu, Z. Shao, J. Jia, and N. Gong, “PromptLocate: Localizing Prompt Injection Attacks,” Oct. 17, 2025, arXiv: arXiv:2510.12252. doi: 10.48550/arXiv.2510.12252.

[20] D. Jacob, H. Alzahrani, Z. Hu, B. Alomair, and D. Wagner, “PromptShield: Deployable Detection for Prompt Injection Attacks,” Apr. 12, 2025, arXiv: arXiv:2501.15145. doi: 10.48550/arXiv.2501.15145.

[21] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” May 23, 2023, arXiv: arXiv:2305.14314. doi: 10.48550/arXiv.2305.14314.

[22] Y. Sheng et al., “S-LoRA: Serving Thousands of Concurrent LoRA Adapters,” Jun. 05, 2024, arXiv: arXiv:2311.03285. doi: 10.48550/arXiv.2311.03285.

[23] X. Suo, “Signed-Prompt: A New Approach to Prevent Prompt Injection Attacks Against LLM-Integrated Applications,” Jan. 15, 2024, arXiv: arXiv:2401.07612. doi: 10.48550/arXiv.2401.07612.

[24] S. Subramanian, V. Elango, and M. Gungor, “Small Language Models (SLMs) Can Still Pack a Punch: A survey,” Jan. 03, 2025, arXiv: arXiv:2501.05465. doi: 10.48550/arXiv.2501.05465.

[25] P. Belcak et al., “Small Language Models are the Future of Agentic AI,” Sep. 15, 2025, arXiv: arXiv:2506.02153. doi: 10.48550/arXiv.2506.02153.

[26] R. Sharma and M. Mehta, “Small Language Models for Agentic Systems: A Survey of Architectures, Capabilities, and Deployment Trade offs,” Oct. 04, 2025, arXiv: arXiv:2510.03847. doi: 10.48550/arXiv.2510.03847.

[27] L. Li, L. Sleem, N. Gentile, G. Nichil, and R. State, “Small Language Models in the Real World: Insights from Industrial Text Classification,” Jun. 23, 2025, arXiv: arXiv:2505.16078. doi: 10.48550/arXiv.2505.16078.

[28] T. H. Sakib, M. T. Hosain, and M. K. Morol, “Small Language Models: Architectures, Techniques, Evaluation, Problems and Future Adaptation,” May 29, 2025, arXiv: arXiv:2505.19529. doi: 10.48550/arXiv.2505.19529.

[29] Q. Zhang, Z. Liu, and S. Pan, “The Rise of Small Language Models,” IEEE Intell. Syst., vol. 40, no. 1, pp. 30–37, Jan. 2025, doi: 10.1109/MIS.2024.3517792.

[30] P. Zhang, G. Zeng, T. Wang, and W. Lu, “TinyLlama: An Open-Source Small Language Model,” Jun. 04, 2024, arXiv: arXiv:2401.02385. doi: 10.48550/arXiv.2401.02385.

[31] H. Lin, Y. Lao, T. Geng, T. Yu, and W. Zhao, “UniGuardian: A Unified Defense for Detecting Prompt Injection, Backdoor Attacks and Adversarial Attacks in Large Language Models,” Feb. 18, 2025, arXiv: arXiv:2502.13141. doi: 10.48550/arXiv.2502.13141.